

Secrets Management Cheat Sheet

1 Introduction

Secrets are being used everywhere nowadays, especially with the popularity of the DevOps movement. Application Programming Interface (API) keys, database credentials, Identity and Access Management (IAM) permissions, Secure Shell (SSH) keys, certificates, etc. Many organizations have them hardcoded within the source code in plaintext, littered throughout configuration files and configuration management tools.

There is a growing need for organizations to centralize the storage, provisioning, auditing, rotation and management of secrets to control access to secrets and prevent them from leaking and compromising the organization. Often, services share the same secrets, which makes identifying the source of compromise or leak challenging.

This cheat sheet offers best practices and guidelines to help properly implement secrets management.

2 General Secrets Management

The following sections address the main concepts relating to secrets management.

2.1 High Availability

It is vital to select a technology that is robust enough to service traffic reliably:

- Users (e.g., SSH keys, root account passwords). In an incident response scenario, users expect to be provisioned with credentials rapidly, so they can recover services that have gone offline. Having to wait for credentials could impact the responsiveness of the operations team.
- Applications (e.g., database credentials and API keys). If the service is not performant, it could degrade the availability of dependent applications or increase application startup times.

Such a service could receive a considerable volume of requests within a large organization.

2.2 Centralize and Standardize

Secrets used by your DevOps teams for your applications might be consumed differently than secrets stored by your marketeers or your SRE team. You often find poorly maintained secrets where the needs of secret consumers or producers mismatch. Therefore, you must standardize and centralize the secrets management solution with care. Standardizing and centralizing can mean that you use multiple secret management solutions. For instance: your cloud-native development teams choose to use the solution provided by the cloud provider, while your private cloud uses a third-party solution, and everybody has an account for a selected password manager. By making sure that the teams standardize the interaction with these different solutions, they remain maintainable and usable in the event of an incident. Even when a company centralizes its secrets management to just one solution, you will often have to secure the primary secret of that secrets management solution in a secondary secrets management solution. For instance, you can use a cloud provider's facilities to store secrets, but that cloud provider's root/management credentials need to be stored somewhere else.

Standardization should include Secrets life cycle management, Authentication, Authorization, and Accounting of the secrets management solution, and life cycle management. Note that it should be immediately apparent to an organization what a secret is used for and where to find it. The more Secrets management solutions you use, the more documentation you need.

2.3 Access Control

When users can read and/or update the secret in a secret management system, it means that the secret can now leak through that user and the system they used to touch the secret. Therefore, engineers should not have access to all secrets in the secrets management system, and the Least Privilege principle should be applied. The secret management system needs to provide the ability to configure fine-grained access controls on each object and component to accomplish the Least Privilege principle.

2.4 Automate Secrets Management

Manual maintenance not only increases the risk of leakage; it also introduces the risk of human errors while maintaining the secret. Furthermore, it can become wasteful. Therefore, it is better to limit or remove the human interaction with the actual secrets. You can restrict human interaction in multiple ways:

- **Secrets pipeline:** Having a secrets pipeline that does large parts of the secret management (e.g., creation, rotation, etc.)
- **Using dynamic secrets:** When an application starts, it could request its database credentials, which, when dynamically generated, will be provided with new credentials for that session. Dynamic secrets should be used where possible to reduce the surface area of credential

reuse. Should the application's database credentials be stolen, upon reboot they would be expired.

- **Automated rotation of static secrets:** Key rotation is a challenging process when implemented manually, and can lead to mistakes. It is therefore better to automate the rotation of keys or at least ensure that the process is sufficiently supported by IT.

Rotating certain keys, such as encryption keys, might trigger full or partial data re-encryption.

Different strategies for rotating keys exist:

- Gradual rotation
- Introducing new keys for Write operations
- Leaving old keys for Read operations
- Rapid rotation
- Scheduled rotation
- and more...

2.4.1 Architectural Patterns for Automated Rotation

To illustrate how to design systems that support automated secret rotation, here are a few architectural patterns:

EXAMPLE 1: KUBERNETES WITH A SIDECAR CONTAINER

In a Kubernetes environment, a common pattern is to use a sidecar container that is responsible for retrieving secrets from a secrets manager and making them available to the main application container. This decouples the application from the specifics of the secrets management solution.

- **Architecture:**
 - A Pod contains two containers: the main application container and a sidecar container (e.g., HashiCorp Vault Agent, CyberArk Conjur Secrets Provider).
 - The sidecar container authenticates with the secrets manager (e.g., using a Kubernetes Service Account).
 - It retrieves the secret and writes it to a shared in-memory volume.
 - The application container reads the secret from the shared volume.
 - The sidecar container can periodically refresh the secret, ensuring the application always has a valid, short-lived credential.
- **Kubernetes Manifest Snippet:**

```

apiVersion: v1
kind: Pod
metadata:
  name: my-app
spec:
  serviceAccountName: my-app-sa
  containers:
    - name: my-app-container
      image: my-app-image
      volumeMounts:
        - name: secrets-volume
          mountPath: "/mnt/secrets"
          readOnly: true
    - name: vault-agent-sidecar
      image: vault:latest
      args: ["agent", "-config=/etc/vault/vault-agent-config.hcl"]
      volumeMounts:
        - name: secrets-volume
          mountPath: "/mnt/secrets"
  volumes:
    - name: secrets-volume
      emptyDir:
        medium: "Memory"

```

EXAMPLE 2: SERVERLESS FUNCTION FOR DATABASE CREDENTIAL ROTATION

Cloud-native secret managers often provide built-in support for automated rotation using serverless functions (e.g., AWS Lambda, Azure Functions).

- **Architecture:**

- A secret is stored in a cloud secrets manager (e.g., AWS Secrets Manager).
- The secrets manager is configured to trigger a rotation Lambda function on a schedule.
- The Lambda function has the necessary permissions to update the database password and the secret value in the secrets manager.
- The rotation process typically involves multiple steps (create new secret, set new secret, test new secret, finish rotation) to ensure a safe transition.

- **AWS Lambda Rotation Function (Conceptual Python Code):**

```

import boto3
import os

def lambda_handler(event, context):
    secret_name = event['SecretId']
    token = event['ClientRequestToken']
    step = event['Step']

```

```
secrets_manager = boto3.client('secretsmanager')
# Get the secret metadata
metadata = secrets_manager.describe_secret(SecretId=secret_name)

if step == "createSecret":
    # Create a new version of the secret
    new_password = generate_new_password()
    secrets_manager.put_secret_value(
        SecretId=secret_name,
        ClientRequestToken=token,
        SecretString=f'{{"password":"{new_password}"}}',
        VersionStages=[ 'AWSPENDING' ]
    )
elif step == "setSecret":
    # Update the database with the new password
    update_database_password(new_password)
elif step == "testSecret":
    # Test the new secret
    test_database_connection(new_password)
elif step == "finishSecret":
    # Mark the new version of the secret as current
    secrets_manager.update_version_stage(
        SecretId=secret_name,
        VersionStage="AWSCURRENT",
        MoveToVersionId=token
    )
```

These examples demonstrate how you can create architectures that not only manage secrets securely but also automate the rotation process, significantly reducing the risk of compromised credentials.

2.5 Handling Secrets in Memory

An additional level of security can be achieved by minimizing the time window where a secret is in memory and limiting the access to its memory space.

Depending on your application's particular circumstances, this can be difficult to implement in a manner that ensures memory security. Because of this potential implementation complexity, you are first encouraged to develop a threat model in order to clearly surface your implicit assumptions about both your application's deployment environment as well as understand the capabilities of your adversaries.

Often attempting to protect secrets in memory will be considered overkill because as you evaluate a threat model, the potential threat actors that you consider either do not have the capabilities to carry out such attacks or the cost of defense far exceeds the likely impact of a compromise arising from exposing secrets in memory. Also, it should be kept in mind while developing an appropriate

threat model, that if an attacker already has access to the memory of the process handling the secret, by that time a security breach may have already occurred. Furthermore, it should be recognized that with the advent of attacks like [Rowhammer](#), or [Meltdown and Spectre](#), it is important to understand that the operating system alone is not sufficient to protect your process memory from these types of attacks. This becomes especially important when your application is deployed to the cloud. The only foolproof approach to protecting memory against these and similar attacks is to fully physically isolate your process memory from all other untrusted processes.

Despite the implementation difficulties, in highly sensitive environments, protecting secrets in memory can be a valuable additional layer of security. For example, in scenarios where an advanced attacker can cause a system to crash and gain access to a memory dump, they may be able to extract secrets from it. Therefore, carefully safeguarding secrets in memory is recommended for untrusted environments or situations where tight security is of utmost importance.

Furthermore, in lower-level languages like C/C++, it is relatively easy to protect secrets in memory. Thus, it may be worthwhile to implement this practice even if the risk of an attacker gaining access to the memory is low. On the other hand, for programming languages that rely on garbage collection, securing secrets in memory generally is much more difficult.

- **Structures and Classes:** In .NET and Java, do not use immutable structures such as Strings to store secrets, since it is impossible to force them to be garbage collected. Instead, use primitive types such as byte arrays or char arrays, where the memory can be directly overwritten.
- **Zeroing Memory:** After a secret has been used, the memory it occupied should be zeroed out to prevent it from lingering in memory where it could potentially be accessed.
- **Memory Encryption:** In some cases, it may be possible to use hardware or operating system features to encrypt the entire memory space of the process handling the secret. This can provide an additional layer of security.

Remember, the goal is to minimize the time window where the secret is in plaintext in memory as much as possible.

For more detailed information, see [Testing Memory for Sensitive Data](#) from the OWASP MAS project.

2.6 Auditing

Auditing is an essential part of secrets management due to the nature of the application. You must implement auditing securely to be resilient against attempts to tamper with or delete the audit

logs. At a minimum, you should audit the following:

- Who requested a secret and for what system and role.
- Whether the secret request was approved or rejected.
- When the secret was used and by whom/what.
- When the secret has expired.
- Whether there were any attempts to reuse expired secrets.
- If there have been any authentication or authorization errors.
- When the secret was updated and by whom/what.
- Any administrative actions and possible user activity on the underlying supporting infrastructure stack.

It is essential that all auditing has correct timestamps. Therefore, the secret management solution should have proper time sync protocols set up at its supporting infrastructure. You should monitor the stack on which the solution runs for possible clock-skew and manual time adjustments.

2.7 Secret Lifecycle

Secrets follow a lifecycle. The stages of the lifecycle are as follows:

- Creation
- Rotation
- Revocation
- Expiration

2.7.1 Creation

New secrets must be securely generated and cryptographically robust enough for their purpose. Secrets must have the minimum privileges assigned to them to enable their required use/role.

You should transmit credentials securely, such that ideally, you don't send the password along with the username when requesting user accounts. Instead, you should send the password via a secure channel (e.g., mutually authenticated connection) or a side-channel such as push notification, SMS, email. Refer to the [Multi-Factor Authentication Cheat Sheet](#) to learn about the pros and cons of each channel.

Applications may not benefit from having multiple communication channels, so you must provision credentials securely.

See [the Open CRE project on secrets lookup](#) for more technical recommendations on secret creation.

2.7.2 Rotation

You should regularly rotate secrets so that any stolen credentials will only work for a short time. Regular rotation will also reduce the tendency for users to fall back to bad habits such as reusing credentials.

Depending on a secret's function and what it protects, the lifetime could be from minutes (think end-to-end encrypted chats with perfect forward secrecy) to years (consider hardware secrets).

User credentials are excluded from regular rotation. These should only be rotated if there is suspicion or evidence that they have been compromised, according to [NIST recommendations](#).

2.7.3 Revocation

When secrets are no longer required or potentially compromised, you must securely revoke them to restrict access. With (TLS) certificates, this also involves certificate revocation.

2.7.4 Expiration

You should create secrets to expire after a defined time where possible. This expiration can either be active expiration by the secret consuming system, or an expiration date set at the secrets management system forcing supporting processes to be triggered, resulting in a secret rotation. You should apply policies through the secrets management solution to ensure credentials are only made available for a limited time appropriate for the type of credentials. Applications should verify that the secret is still active before trusting it.

2.8 Transport Layer Security (TLS) Everywhere

Never transmit secrets via plaintext. In this day and age, there is no excuse given the ubiquitous adoption of TLS.

Furthermore, you can effectively use secrets management solutions to provision TLS certificates.

2.9 Downtime, Break-glass, Backup and Restore

Consider the possibility that a secrets management service becomes unavailable for various reasons, such as scheduled downtime for maintenance. It could be impossible to retrieve the credentials required to restore the service if you did not previously acquire them. Thus, choose maintenance windows carefully based on earlier metrics and audit logs.

Next, the backup and restore procedures of the system should be regularly tested and audited for their security. A few requirements regarding backup & restore. Ensure that:

- An automated backup procedure is in place and executed periodically; base the frequency of the backups and snapshots on the number of secrets and their lifecycle.
- Frequently test restore procedures to guarantee that the backups are intact.
- Encrypt backups and put them on secure storage with reduced access rights. Monitor the backup location for (unauthorized) access and administrative actions.

Lastly, you should implement emergency ("break-glass") processes to restore the service if the system becomes unavailable for reasons other than regular maintenance. Therefore, emergency break-glass credentials should be regularly backed up securely in a secondary secrets management system and tested routinely to verify they work.

2.10 Policies

Consistently enforce policies defining the minimum complexity requirements of passwords and approved encryption algorithms at an organization-wide level. Using a centralized secrets management solution can help companies implement these policies.

Next, having an organization-wide secrets management policy can help enforce applying the best practices defined in this cheat sheet.

2.11 Metadata: prepare to move the secret

A secret management solution should provide the capability to store at least the following metadata about a secret:

- When it was created/consumed/archived/rotated/deleted
- Who created/consumed/archived/rotated/deleted it (e.g., both the actual producer and the engineer using the production method)
- What created/consumed/archived/rotated/deleted it
- Who to contact when having trouble with the secret or having questions about it
- For what the secret is used (e.g., designated intended consumers and purpose of the secret)
- What type of secret it is (e.g., AES Key, HMAC key, RSA private key)
- When you need to rotate it, if done manually

Note: if you don't store metadata about the secret nor prepare to move, you will increase the probability of vendor lock-in.

2.12 Passwordless Authentication and Token Security

While not a direct replacement for all types of secrets (e.g., API keys, database credentials), passwordless authentication mechanisms like **OpenID Connect (OIDC)** can significantly reduce the attack surface by moving away from user-managed passwords. Instead of passwords, applications rely on trusted identity providers (IdPs) to authenticate users and receive secure tokens.

How it helps:

- **Reduces Password-Related Risks:** Eliminates threats like phishing, credential stuffing, and weak password practices.
- **Centralized Identity Management:** Authentication is handled by a specialized IdP, which can enforce strong authentication policies (e.g., MFA).
- **Short-Lived Sessions:** OIDC tokens are typically short-lived, limiting the window of opportunity for an attacker if a token is compromised.

Token Security is Crucial:

Adopting passwordless authentication shifts the security focus from protecting static passwords to protecting dynamic tokens (e.g., ID tokens, access tokens, refresh tokens). These tokens are bearer tokens, meaning anyone who possesses one can use them. Therefore, it is critical to:

- **Secure Token Transmission:** Always transmit tokens over TLS.
- **Protect Tokens in Storage:** Do not store tokens in insecure locations like local storage in a browser. Use secure, HTTP-only cookies or appropriate secure storage mechanisms for mobile applications.
- **Validate Tokens Correctly:** Always validate the signature, issuer, and audience of a token to ensure it is legitimate.
- **Manage Token Lifetime:** Use short-lived access tokens and implement a secure refresh token rotation strategy.

For more detailed guidance on securing OAuth 2.0 and OpenID Connect implementations, refer to the [OAuth2 Cheat Sheet](#).

3 Continuous Integration (CI) and Continuous Deployment (CD)

Building, testing and deploying changes generally requires access to many systems. Continuous Integration (CI) and Continuous Deployment (CD) tools typically store secrets to provide configuration to the application or during deployment. Alternatively, they interact heavily with the

secrets management system. Various best practices can help smooth out secret management in CI/CD; we will deal with some of them in this section.

3.1 Hardening your CI/CD pipeline

CI/CD tooling consumes (high-privilege) credentials regularly. Ensure that the pipeline cannot be easily hacked or misused by employees. Here are a few guidelines which can help you:

- Treat your CI/CD tooling as a production environment: harden it, patch it and harden the underlying infrastructure and services.
- Have Security Event Monitoring in place.
- Implement least-privilege access: developers do not need to be able to administer projects. Instead, they only need to be able to execute required functions, such as setting up pipelines, running them, and working with code. Administrative tasks can quickly be done using configuration-as-code in a separate repository used by the CI/CD system to update its configuration. There is no need for privileged roles that might have access to secrets.
- Make sure that pipeline output does not leak secrets, and you can't listen in on production pipelines with debugging tools.
- Make sure you cannot exec into any runners and workers for a CI/CD system.
- Have proper authentication, authorization and accounting in place.
- Ensure only an approved process can create pipelines, including MR/PR steps to ensure that a created pipeline is security-reviewed.

3.2 Where should a secret be?

There are various places where you can store a secret to execute CI/CD actions:

- As part of your CI/CD tooling: you can store a secret in [GitLab](#)/[GitHub](#)/[Jenkins](#). This is not the same as committing it to code.
- As part of your secrets-management system: you can store a secret in a secrets management system, such as facilities provided by a cloud provider ([AWS Secrets Manager](#), [Azure Key Vault](#), [Google Secret Manager](#)), or other third-party facilities ([Hashicorp Vault](#), [Conjur](#), [Keeper](#)). In this case, the CI/CD pipeline tooling requires credentials to connect to these secret management systems to have secrets in place. See [Cloud Providers](#) for more details on using a cloud provider's secret management system.

Another alternative here is using the CI/CD pipeline to leverage the Encryption as a Service from the secrets management systems to do the encryption of a secret. The CI/CD tooling can then

commit the encrypted secret to git, which can be fetched by the consuming service on deployment and decrypted again. See section 3.6 for more details.

Note: not all secrets must be in the CI/CD pipeline to get to the actual deployment. Instead, make sure that the deployed services take care of part of their secrets management at their own lifecycle (e.g., deployment, runtime and destruction).

3.2.1 As part of your CI/CD tooling

When secrets are part of your CI/CD tooling, it means that these secrets are exposed to your CI/CD jobs. CI/CD tooling can comprise, e.g., GitHub secrets, GitLab repository secrets, ENV Vars/Var Groups in Microsoft Azure DevOps, Kubernetes Secrets, etc. These secrets are often configurable/viewable by people who have the authorization to do so (e.g., a maintainer in GitHub, a project owner in GitLab, an admin in Jenkins, etc.), which together line up for the following best practices:

- No "big secret": ensure that secrets in your CI/CD tooling that are not long-term, don't have a wide blast radius, and don't have a high value. Also, limit shared secrets (e.g., never have one password for all administrative users).
- As is / To be: have a clear overview of which users can view or alter the secrets. Often, maintainers of a GitLab/GitHub project can see or otherwise extract its secrets.
- Reduce the number of people that can perform administrative tasks on the project to limit exposure.
- Log & Alert: Assemble all the logs from the CI/CD tooling and have rules in place to detect secret extraction or misuse, whether through accessing them through a web interface or dumping them while double Base64 encoding or encrypting them with OpenSSL.
- Rotation: Regularly rotate secrets.
- Forking should not leak: Validate that a fork of the repository or copy of the job definition does not copy the secret.
- Document: Make sure you document which secrets you store as part of your CI/CD tooling and why so that you can migrate these easily when required.

3.2.2 Storing it in a secrets management system

Naturally, you can store secrets in a designated secrets management solution. For example, you can use a solution offered by your (cloud) infrastructure provider, such as [AWS Secrets Manager](#), [Google Secrets Manager](#), or [Azure Key Vault](#). You can find more information about these in [section 4](#) of this cheat sheet. Another option is a dedicated secrets management system, such as [Hashicorp Vault](#), [Keeper](#), [Conjur](#). Here are a few do's and don'ts for the CI/CD interaction with these systems. Make sure that the following is taken care of:

- Rotation/Temporality: credentials used by the CI/CD tooling to authenticate against the secret management system are rotated frequently and expire after a job completes.
- Scope of authorization: scope credentials used by the CI/CD tooling (e.g., roles, users, etc.), only authorize those secrets and services of the secret management system required for the CI/CD tooling to execute its job.
- Attribution of the caller: credentials used by the CI/CD tooling still hold attribution of the one calling the secrets management solution. Ensure you can attribute any calls made by the CI/CD tooling to a person or service that requested the actions of the CI/CD tooling. If this is not possible through the default configuration of the secrets manager, make sure that you have a correlation setup in terms of request parameters.
- All of the above: Still follow those do's and don'ts listed in section 3.2.1: log & alert, take care of forking, etc.
- Backup: back up secrets to product-critical operations in separate storage (e.g., cold storage), especially encryption keys.

3.2.3 Not touched by CI/CD at all

Secrets do not necessarily need to be brought to a consumer of the secret by a CI/CD pipeline. It is even better when the consumer of the secret retrieves the secret. In that case, the CI/CD pipeline still needs to instruct the orchestrating system (e.g., [Kubernetes](#)) that it needs to schedule a specific service with a given service account with which the consumer can then retrieve the required secret. The CI/CD tooling then still has credentials for the orchestrating platform but no longer has access to the secrets themselves. The do's and don'ts regarding these credentials types are similar to those described in section 3.2.2.

3.3 Authentication and Authorization of CI/CD tooling

CI/CD tooling should have designated service accounts, which can only operate in the scope of the required secrets or orchestration of the consumers of a secret. Additionally, a CI/CD pipeline run should be easily attributable to the one who has defined the job or triggered it to detect who has tried to exfiltrate secrets or manipulate them. When you use certificate-based auth, the caller of the pipeline identity should be part of the certificate. If you use a token to authenticate towards the mentioned systems, make sure you set the principal requesting these actions (e.g., the user or the job creator).

Verify on a periodic basis whether this is (still) the case for your system so that you can do logging, attribution, and security alerting on suspicious actions effectively.

3.4 Logging and Accounting

Attackers can use CI/CD tooling to extract secrets. They could, for example, use administrative interfaces or job creation that exfiltrates the secret using encryption or double Base64 encoding. Therefore, you should log every action in a CI/CD tool. You should define security alerting rules at every non-standard manipulation of the pipeline tool and its administrative interface to monitor secret usage. Logs should be queryable for at least 90 days and stored for a more extended period in cold storage. It might take security teams time to understand how attackers can exfiltrate or manipulate a secret using CI/CD tooling.

3.5 Rotation vs Dynamic Creation

You can leverage CI/CD tooling to rotate secrets or instruct other components to do the rotation of the secret. For instance, the CI/CD tool can request a secrets management system or another application to rotate the secret. Alternatively, the CI/CD tool or another component could set up a dynamic secret: a secret required for a consumer to use for as long as it lives. The secret is invalidated when the consumer no longer lives. This procedure reduces possible leakage of a secret and allows for easy detection of misuse. If an attacker uses a secret from anywhere other than the consumer's IP, you can easily detect it.

3.6 Pipeline Created Secrets

You can use pipeline tooling to generate secrets and either offer them directly to the service deployed by the tooling or provide the secret to a secrets management solution. Alternatively, the secret can be stored encrypted in git so that the secret and its metadata is as close to the developer's daily place of work as possible. A git-stored secret does require that developers cannot decrypt the secrets themselves and that every consumer of a secret has its encrypted variant of the secret. For instance: the secret should then be different per DTAP environment and be encrypted with another key. For each environment, only the designated consumer in that environment should be able to decrypt the specific secret. A secret does not leak cross-environment and can still be easily stored next to the code. Consumers of a secret could now decrypt the secret using a sidecar, as described in section 5.2. Instead of retrieving the secrets, the consumer would leverage the sidecar to decrypt the secret.

When a pipeline creates a secret by itself, ensure that the scripts or binaries involved adhere to best practices for secret generation. Best practices include secure randomness, proper length of secret creation, etc. and that the secret is created based on well-defined metadata stored somewhere in git or somewhere else.

4 Cloud Providers

For cloud providers, there are at least four essential topics to touch upon:

- Designated secret storage/management solutions. Which service(s) do you use?
- Envelope & client-side encryption
- Identity and access management: decreasing the blast radius
- API quotas or service limits

4.1 Services to Use

It is best to use a designated secret management solution in any environment. Most cloud providers have at least one service that offers secret management. Of course, it's also possible to run a different secret management solution (e.g., HashiCorp Vault or Conjur) on compute resources within the cloud. We'll consider cloud provider service offerings in this section.

Sometimes it's possible to automatically rotate your secret, either via a service provided by your cloud provider or a (custom-built) function. Generally, you should prefer the cloud provider's solution since the barrier of entry and risk of misconfiguration are lower. If you use a custom solution, ensure the function's role to do its rotation can only be assumed by said function.

4.1.1 AWS

For AWS, the recommended solution is [AWS Secrets Manager](#).

Permissions are granted at the secret level. Check out the [Secrets Manager best practices](#).

It is also possible to use the [Systems Manager Parameter Store](#), which is cheaper, but that has a few downsides:

- you'll need to make sure you've specified encryption yourself (secrets manager does that by default)
- it offers fewer auto-rotation capabilities (you will likely need to build a custom function)
- it doesn't support cross-account access
- it doesn't support cross-region replication
- there are fewer [Security Hub Controls](#) available

4.1.1.1 AWS NITRO ENCLAVES

With [AWS Nitro Enclaves](#), you can create isolated compute environments to further protect and securely process highly sensitive data such as secrets. Enclaves are hardened, and restrict operator access, providing a trusted execution environment. A key feature is cryptographic attestation, which allows you to verify the enclave's identity and ensure only authorized code is

running before provisioning secrets to it. This makes it a strong choice for scenarios requiring high assurance in secret handling.

4.1.1.2 AWS CLOUDHSM

For secrets being used in highly confidential applications, it may be needed to have more control over the encryption and storage of these keys. AWS offers [CloudHSM](#), which lets you bring your own key (BYOK) for AWS services. Thus, you will have more control over keys' creation, lifecycle, and durability. CloudHSM allows automatic scaling and backup of your data. The cloud service provider, Amazon, will not have any access to the key material stored in **AWS CloudHSM**.

4.1.2 GCP

For GCP, the recommended service is [Secret Manager](#).

Permissions are granted at the secret level.

Check out the [Secret Manager best practices](#).

4.1.2.1 GOOGLE CLOUD CONFIDENTIAL COMPUTING

[GCP Confidential Computing](#) is a technology that encrypts data in-use, while it is being processed. This is achieved through services like **Confidential VMs** and **Confidential GKE Nodes**, which leverage AMD Secure Encrypted Virtualization (SEV). This ensures that even Google personnel cannot view the contents of the memory of your virtual machines, providing a high degree of protection for secrets that must be held in memory.

4.1.3 Azure

For Azure, the recommended service is [Key Vault](#).

Contrary to other clouds, permissions are granted at the **Key Vault** level. This means secrets for separate workloads and separate sensitivity levels should be in separated Key Vaults accordingly.

Check out the [Key Vault best practices](#).

4.1.3.1 AZURE CONFIDENTIAL COMPUTING

With [Azure Confidential Computing](#), you can create trusted execution environments. This technology isolates sensitive data within a protected container, ensuring that it is encrypted both at rest, in transit, and in use. Services like **Azure Confidential Virtual Machines** and **Confidential Containers on ACI** utilize technologies like Intel SGX and AMD SEV-SNP to create these secure enclaves. This prevents unauthorized access from cloud administrators, malware, or other tenants, making it a robust solution for secret management.

4.1.3.2 AZURE DEDICATED HSM

For secrets being used in Azure environments and requiring special security considerations, Azure offers [Azure Dedicated HSM](#). This allows you more control over the secrets stored on it, including enhanced administrative and cryptographic control. The cloud service provider, Microsoft, will not have any access to the key material stored in Azure Dedicated HSM.

4.1.4 Other clouds, Multi-cloud, and Cloud agnostic

If you're using multiple cloud providers, you should consider using a cloud-agnostic secret management solution. This will allow you to use the same secret management solution across all your cloud providers (and possibly also on-premises). Another advantage is that this avoids vendor lock-in with a specific cloud provider, as the solution can be used on any cloud provider.

There are open-source and commercial solutions available. Some examples are:

- [CyberArk Conjur](#)
- [HashiCorp Vault](#)
- [Pulumi ESC](#)

4.2 Envelope & client-side encryption

This section will describe how a secret is encrypted and how you can manage the keys for that encryption in the cloud.

4.2.1 Client-side encryption versus server-side encryption

Server-side encryption of secrets ensures that the cloud provider takes care of the encryption of the secret in storage. The secret is then safeguarded against compromise while at rest. Encryption at rest often does not require additional work other than selecting the key to encrypt it with (See section 4.2.2). However, when you submit the secret to another service, it will no longer be encrypted. It is decrypted before sharing with the intended service or human user.

Client-side encryption of secrets ensures that the secret remains encrypted until you actively decrypt it. This means it is only decrypted when it arrives at the consumer. You need to have a proper crypto system to cater for this. Think about mechanisms such as PGP using a safe configuration and other more scalable and relatively easy to use systems. Client-side encryption can provide an end-to-end encryption of the secret: from producer to consumer.

4.2.2 Bring Your Own Key versus Cloud Provider Key

When you encrypt a secret at rest, the question is: which key do you want to use? The less trust you have in the cloud provider, the more you will want to manage yourself.

Often, you can either encrypt a secret with a key managed at the secrets management service or use a key management solution from the cloud provider to encrypt the secret. The key offered through the key management solution of the cloud provider can be either managed by the cloud provider or by yourself. Industry standards call the latter "bring your own key" (BYOK). You can either directly import or generate this key at the key management solution or using cloud HSM supported by the cloud provider. You can then either use your key or the customer main key from the provider to encrypt the data key of the secrets management solution. The data key, in turn, encrypts the secret. By managing the CMK, you have control over the data key at the secrets management solution.

While importing your own key material can generally be done with all providers ([AWS](#), [Azure](#), [GCP](#)), unless you know what you are doing and your threat model and policy require this, this is not a recommended solution due to its complexity and difficulty of use.

4.3 Identity and Access Management (IAM)

IAM applies to both on-premises and cloud setups: to effectively manage secrets, you need to set up suitable access policies and roles. Setting this up goes beyond policies regarding secrets; it should include hardening the full IAM setup, as it could otherwise allow for privilege escalation attacks. Ensure you never allow open "pass role" privileges or unrestricted IAM creation privileges, as these can use or create credentials that have access to the secrets. Next, make sure you tightly control what can impersonate a service account: are your machines' roles accessible by an attacker exploiting your server? Can service roles from the data-pipeline tooling access the secrets easily? Ensure you include IAM for every cloud component in your threat model (e.g., ask yourself: how can you do elevation of privileges with this component?). See [this blog entry](#) for multiple do's and don'ts with examples.

Leverage the temporality of the IAM principals effectively: e.g., ensure that only specific roles and service accounts that require it can access the secrets. Monitor these accounts so that you can tell who or what used them to access the secrets.

Next, make sure that you scope access to your secrets: one should not be simply allowed to access all secrets. In GCP and AWS, you can create fine-grained access policies to ensure that a principal cannot access all secrets at once. In Azure, having access to the key vault means having access to all secrets in that key vault. It is, thus, essential to have separate key vaults when working on Azure to segregate access.

4.4 API limits

Cloud services can generally provide a limited amount of API calls over a given period. You could potentially (D)DoS yourself when you run into these limits. Most of these limits apply per account, project, or subscription, so spread workloads to limit your blast radius accordingly. Additionally, some services may support data key caching, preventing load on the key management service API (see, for example, [AWS data key caching](#)). Some services can leverage built-in data key caching. [S3](#) is one such example.

5 Containers & Orchestrators

You can enrich containers with secrets in multiple ways: build time (not recommended) and during orchestration/deployment.

5.1 Injection of Secrets (file, in-memory)

There are three ways to get secrets to an app inside a Docker container.

- Mounted volumes (file): With this method, we keep our secrets within a particular config/secret file and mount that file to our instance as a mounted volume. Ensure that these mounts are mounted in by the orchestrator and never built-in, as this will leak the secret with the container definition. Instead, make sure that the orchestrator mounts in the volume when required.
- Fetch from the secret store (in-memory): A sidecar app/container fetches the secrets it needs directly from a secret manager service without dealing with docker config. This solution allows you to use dynamically constructed secrets without worrying about the secrets being viewable from the file system or from checking the Docker container's environment variables.
- Environment variables: We can provide secrets directly as part of the Docker container configuration. Note: secrets themselves should never be hardcoded using docker ENV or docker ARG commands, as these can easily leak with the container definitions. See the Docker challenges at [WrongSecrets](#) as well. Instead, let an orchestrator overwrite the environment variable with the actual secret and ensure that this is not hardcoded. Additionally, environment variables are generally accessible to all processes and may be included in logs or system dumps. Using environment variables is therefore not recommended unless the other methods are not possible.

5.2 Short-Lived Sidecar Containers

To inject secrets, you could create short-lived sidecar containers that fetch secrets from some remote endpoint and then store them on a shared volume mounted to the original container. The original container can now use the secrets from the mounted volume. The benefit of using this

approach is that we don't need to integrate any third-party tool or code to get secrets. Once the sidecar has fetched the secrets, it terminates. Examples of this include [Vault Agent Sidecar Injector](#) and [Conjur Secrets Provider](#). By mounting secrets to a volume shared with the pod, containers within the pod can consume secrets without being aware of the secrets manager.

5.3 Internal vs External Access

You should only expose secrets to communication mechanisms between the container and the deployment representation (e.g., a Kubernetes Pod). Never expose secrets through external access mechanisms shared among deployments or orchestrators (e.g., a shared volume).

When the orchestrator stores secrets (e.g., Kubernetes Secrets), make sure that the storage backend of the orchestrator is encrypted and you manage the keys well. See the [Kubernetes Security Cheat Sheet](#) for more information.

6 Implementation Guidance

In this section, we will discuss implementation. Note that it is always best to refer to the official documentation of the secrets management system of choice for the actual implementation as it will be more up to date than any secondary document such as this cheat sheet.

6.1 Key Material Management Policies

Key material management is discussed in the [Key Management Cheat Sheet](#)

6.2 Dynamic vs Static Use Cases

We see the following use cases for dynamic secrets, among others:

- short-lived secrets (e.g., credentials or API keys) for a secondary service that expresses the intent for connecting the primary service (e.g., consumer) to the service.
- short-lived integrity and encryption controls for guarding and securing in-memory and runtime communication processes. Think of encryption keys that only need to live for a single session or a single deployment lifetime.
- short-lived credentials for building a stack during the deployment of a service for interacting with the deployers and supporting infrastructure.

Note that these dynamic secrets often need to be created with the service we need to connect to. To create these types of dynamic secrets, we usually require long-term static secrets to create the

dynamic secrets themselves. Other static use cases:

- key material that needs to live longer than a single deployment due to the nature of its usage in the interaction with other instances of the same service (e.g., storage encryption keys, TLS PKI keys)
- key material or credentials to connect to services that do not support creating temporal roles or credentials.

6.3 Ensure limitations are in place

Secrets should never be retrievable by everyone and everything. Always make sure that you put guardrails in place:

- Do you have the opportunity to create access policies? Ensure that there are policies in place to limit the number of entities that can read or write the secret. At the same time, write the policies so that you can easily extend them, and they are not too complicated to understand.
- Is there no way to reduce access to certain secrets within a secrets management solution? Consider separating the production and development secrets by having separate secret management solutions. Then, reduce access to the production secrets management solution.

6.4 Security Event Monitoring is Key

Continually monitor who/what, from which IP, and what methodology accesses the secret. There are various patterns to look out for, such as, but not limited to:

- Monitor who accesses the secret at the secret management system: is this normal behavior? If the CI/CD credentials are used to access the secret management solution from a different IP than where the CI/CD system is running, provide a security alert and assume the secret is compromised.
- Monitor the service requiring the secret (if possible), e.g., whether the user of the secret is coming from an expected IP, with an expected user agent. If not, alert and assume the secret is compromised.

6.5 Usability and Ease of Onboarding

For a secrets management solution to be effective, it must be easy for developers to adopt and use. If the process is too complex, developers may resort to insecure practices. A focus on usability and a smooth onboarding experience is critical.

- **Clear and Comprehensive Documentation:**

- Provide clear, concise, and easy-to-find documentation. This should include tutorials for common use cases, detailed API references, and practical examples.
- Maintain a "getting started" guide that walks new users through the process of obtaining their first secret.
- **Developer-Friendly Tooling and SDKs:**
 - Offer well-maintained SDKs for various programming languages to simplify integration.
 - Provide a command-line interface (CLI) that allows developers to manage secrets from their local development environment.
 - Develop plugins for common IDEs, CI/CD systems, and infrastructure-as-code (IaC) tools like Terraform and Pulumi.
- **Streamlined Workflows:**
 - Implement self-service workflows that enable developers to request and receive secrets with minimal manual intervention.
 - Use GitOps principles to manage secrets as code, allowing developers to define secret needs in a declarative manner alongside their application code.
 - Automate the approval process for low-risk secrets while maintaining appropriate controls for more sensitive ones.
- **Actionable Feedback and Support:**
 - Provide clear error messages that help developers troubleshoot issues independently.
 - Establish dedicated support channels (e.g., a Slack channel, a ticketing system) where developers can get help from the security or platform team.
- **Easy Integration:**
 - Ensure the secrets management solution can be easily integrated with existing applications. Sidecar containers, such as the [Vault Agent Sidecar Injector](#) or the [Conjur Secrets Provider](#), can help decouple applications from the secrets management system.

7 Encryption

Secrets Management goes hand in hand with encryption. After all, secrets must be stored encrypted somewhere to protect their confidentiality and integrity.

7.1 Encryption Types to Use

You can use various encryption types to secure a secret as long as they provide sufficient security, including adequate resistance against quantum computing-based attacks. Given that this is a moving field, it is best to take a look at sources like keylength.com, which enumerate up-to-date recommendations on the usage of encryption types and key lengths for existing standards, as well as the NSA's [Commercial National Security Algorithm Suite 2.0](#) which enumerates quantum resistant algorithms.

Please note that in all cases, we need to preferably select an algorithm that provides encryption and confidentiality at the same time, such as AES-256 using GCM ([Galois Counter Mode](#)), or a mixture of ChaCha20 and Poly1305 according to the best practices in the field.

7.2 Convergent Encryption

[Convergent Encryption](#) ensures that a given plaintext and its key results in the same ciphertext. This can help detect possible reuse of secrets, resulting in the same ciphertext. The challenge with enabling convergent encryption is that it allows attackers to use the system to generate a set of cryptographic strings that might end up in the same secret, allowing the attacker to derive the plaintext secret. Given the algorithm and key, you can mitigate this risk if the convergent crypto system you use has sufficient resource challenges during encryption. Another factor that can help reduce the risk is ensuring that a secret is of adequate length, further hampering the possible guess-iteration time required.

7.3 Where to store the Encryption Keys?

You should not store keys next to the secrets they encrypt, except if those keys are encrypted themselves (see envelope encryption). Start by consulting the [Key Management Cheat Sheet](#) on where and how to store the encryption and possible HMAC keys.

7.4 Encryption as a Service (EaaS)

EaaS is a model in which users subscribe to a cloud-based encryption service without having to install encryption on their own systems. Using EaaS, you can get the following benefits:

- Encryption at rest
- Encryption in transit (TLS)
- Key handling and cryptographic implementations are taken care of by Encryption Service, not by developers
- The provider could add more services to interact with the sensitive data

8 Detection

There are many approaches to secrets detection and some very useful open-source projects to help with this. The [Yelp Detect Secrets](#) project is mature and has signature matching for around 20 secrets. For more information on other tools to help you in the detection space, check out the [Secrets Detection](#) topic on GitHub.

8.1 General detection approaches

Shift-left and DevSecOps principles apply to secrets detection as well. These general approaches below aim to consider secrets earlier and evolve the practice over time.

- Create standard test secrets and use them universally across the organization. This allows for reducing false positives by only needing to track a single test secret for each secret type.
- Consider enabling secrets detection at the developer level to avoid checking secrets into code before commit/PR either in the IDE, as part of test-driven development, or via pre-commit hook.
- Make secrets detection part of the threat model. Consider secrets as part of the attack surface during threat modeling exercises.
- Evaluate detection utilities and related signatures often to ensure they meet expectations.
- Consider having more than one detection utility and correlating/de-duping results to identify potential areas of detection weakness.
- Explore a balance between entropy and ease of detection. Secrets with consistent formats are easier to detect with lower false-positive rates, but you also don't want to miss a human-created password simply because it doesn't match your detection rules.

8.2 Types of secrets to be detected

Many types of secrets exist, and you should consider signatures for each to ensure accurate detection for all. Among the more common types are:

- High availability secrets (Tokens that are difficult to rotate)
- Application configuration files
- Connection strings
- API keys
- Credentials
- Passwords

- 2FA keys
- Private keys (e.g., SSH keys)
- Session tokens
- Platform-specific secret types (e.g., Amazon Web Services, Google Cloud)

For more fun learning about secrets and practice rooting them out, check out the [Wrong Secrets](#) project.

8.3 Detection lifecycle

Secrets are like any other authorization token. They should:

- Exist only for as long as necessary (rotate often)
- Have a method for automatic rotation
- Only be visible to those who need them (least privilege)
- Be revocable (including the logging of attempt to use a revoked secret)
- Never be logged (must implement either an encryption or masking approach in place to avoid logging plaintext secrets)

Create detection rules for each of the stages of the secret lifecycle.

8.4 Documentation for how to detect secrets

Create documentation and update it regularly to inform the developer community on procedures and systems available at your organization and what types of secrets management you expect, how to test for secrets, and what to do in the event of detected secrets.

Documentation should:

- Exist and be updated often, especially in response to an incident
- Include the following information:
 - Who has access to the secret
 - How it gets rotated
 - Any upstream or downstream dependencies that could potentially be broken during secret rotation
 - Who is the point of contact during an incident
 - Security impact of exposure

- Identify when secrets may be handled differently depending on the threat risk, data classification, etc.

9 Incident Response

Quick response in the event of a secret exposure is perhaps one of the most critical considerations for secrets management.

9.1 Documentation

Incident response in the event of secret exposure should ensure that everyone in the chain of custody is aware and understands how to respond. This includes application creators (every member of a development team), information security, and technology leadership.

Documentation must include:

- How to test for secrets and secrets handling, especially during business continuity reviews.
- Whom to alert when a secret is detected.
- Steps to take for containment
- Information to log during the event

9.2 Remediation

The primary goal of incident response is rapid response and containment.

Containment should follow these procedures:

1. Revocation: Keys that were exposed should undergo immediate revocation. The secret must be able to be de-authorized quickly, and systems must be in place to identify the revocation status.
2. Rotation: A new secret must be able to be quickly created and implemented, preferably via an automated process to ensure repeatability, low rate of implementation error, and least-privilege (not directly human-readable).
3. Deletion: Secrets revoked/rotated must be removed from the exposed system immediately, including secrets discovered in code or logs. Secrets in code could have commit history for the exposure squashed to before the introduction of the secret, however, this may introduce other problems as it rewrites git history and will break any other links to a given commit. If you

decide to do this be aware of the consequences and plan accordingly. Secrets in logs must have a process for removing the secret while maintaining log integrity.

4. Logging: Incident response teams must have access to information about the lifecycle of a secret to aid in containment and remediation, including:

- Who had access?
- When did they use it?
- When was it previously rotated?

9.3 Logging

Additional considerations for logging of secrets usage should include:

- Logging for incident response should be to a single location accessible by incident response (IR) teams
- Ensure fidelity of logging information during purple team exercises such as:
 - What should have been logged?
 - What was actually logged?
 - Do we have adequate alerts in place to ensure this?

Consider using a standardized logging format and vocabulary such as the [Logging Vocabulary Cheat Sheet](#) to ensure that all necessary information is logged.

10 Secrets Management in a Multi-Cloud Environment

10.1 Introduction

Managing secrets in a multi-cloud environment presents unique challenges due to the diversity of cloud providers and their respective services. This section discusses the challenges and best practices for managing secrets across multiple cloud providers.

10.2 Challenges

1. **Diverse APIs and Interfaces:** Each cloud provider has its own API and interface for managing secrets, which can lead to complexity in integrating and managing secrets across multiple providers.

2. **Inconsistent Security Policies:** Different cloud providers may have varying security policies and practices, making it challenging to enforce consistent security standards across all environments.
3. **Key Rotation:** Ensuring that keys are rotated consistently and securely across multiple cloud providers can be difficult, especially if each provider has different mechanisms for key rotation.
4. **Access Control:** Managing access control for secrets across multiple cloud providers can be complex, as each provider may have different access control mechanisms and policies.
5. **Auditing and Monitoring:** Ensuring comprehensive auditing and monitoring of secret access and usage across multiple cloud providers can be challenging due to the differences in logging and monitoring capabilities.

10.3 Best Practices

1. **Use a Centralized Secrets Management Solution:** Implement a centralized secrets management solution that can integrate with multiple cloud providers. This can help standardize the management of secrets and enforce consistent security policies across all environments. Examples include HashiCorp Vault and CyberArk Conjur.
2. **Standardize Security Policies:** Define and enforce standardized security policies for managing secrets across all cloud providers. This includes policies for key rotation, access control, and auditing.
3. **Automate Key Rotation:** Implement automated key rotation processes to ensure that keys are rotated consistently and securely across all cloud providers. Use tools and scripts to automate the rotation process and reduce the risk of human error.
4. **Implement Fine-Grained Access Control:** Use fine-grained access control mechanisms to restrict access to secrets based on the principle of least privilege. Ensure that access control policies are consistently enforced across all cloud providers.
5. **Enable Comprehensive Auditing and Monitoring:** Implement comprehensive auditing and monitoring of secret access and usage across all cloud providers. Use centralized logging and monitoring solutions to aggregate and analyze logs from multiple providers.

10.4 References

- [HashiCorp Vault](#)
- [CyberArk Conjur](#)
- [AWS Secrets Manager](#)
- [Azure Key Vault](#)

- [Google Cloud Secret Manager](#)

11 Related Cheat Sheets & further reading

- [Key Management Cheat Sheet](#)
- [Logging Cheat Sheet](#)
- [Password Storage Cheat Sheet](#)
- [Cryptographic Storage Cheat Sheet](#)
- [OWASP WrongSecrets project](#)
- [Blog: 10 Pointers on Secrets Management](#)
- [Blog: From build to run: pointers on secure deployment](#)
- [GitHub listing on secrets detection tools](#)
- [NIST SP 800-57 Recommendation for Key Management](#)
- [OpenCRE References to secrets](#)